RADONAIX — Monitoring Guide (Prometheus + Grafana)

System and application monitoring for the RADONAIX Revenue Assurance platform. This guide covers what we monitor, the dashboards we ship, and how to run the stack both on a laptop (for a quick check) and on the production server.

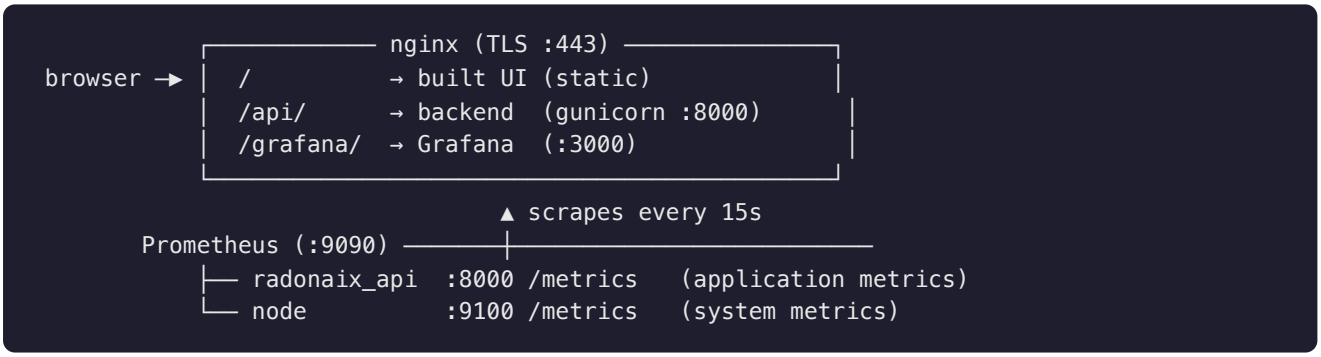
---

1. Overview

We added a lightweight, self-hosted monitoring stack that answers two questions at a glance:

- **Is the server healthy?** — CPU, memory, disk and network of the host.
- **Is the application healthy?** — request rate, error rate and latency of the backend API.

Three small components do the work; everything runs locally on the app server and is reached through the existing HTTPS origin — **no new public ports**.



Component	Port (localhost)	Role
Prometheus	9090	Time-series database; scrapes metrics and stores them (30-day retention)
node_exporter	9100	Exposes host system metrics (CPU, RAM, disk, network)
Grafana	3000	Dashboards / visualisation; reads from Prometheus
Backend `/metrics`	8000	Already built into the app — request count + latency

> The app already exposed Prometheus metrics at `/metrics` before this work; we simply added a collector (Prometheus), a system exporter (node_exporter) and a visualisation layer (Grafana).

---

2. What we monitor

Application metrics (from the backend `/metrics`)

The FastAPI backend publishes these automatically via middleware:

Metric	Type	Labels	Meaning
`http_requests_total`	Counter	`method`, `path`, `status`	Total HTTP requests, broken down by route and status code
`http_request_duration_seconds`	Histogram	`method`, `path`	Request latency distribution (used to compute p50/p95/p99)

Paths use **route templates** (e.g. `/api/users/{id}`) so the data stays compact regardless of traffic.

System metrics (from node_exporter)

Standard host metrics: CPU time per mode, memory available/total, filesystem usage, disk I/O, network throughput, uptime — the same dataset behind the well-known "Node Exporter Full" dashboards.

---

3. The dashboards

Two dashboards are **auto-provisioned** into Grafana under the **RADONAIX** folder — no manual import.

RADONAIX — System

Panel	Shows
CPU busy %	Overall CPU utilisation (gauge, green/amber/red)
Memory used %	RAM in use vs total
Root disk used %	Disk usage on `/`
Uptime	Time since last boot
CPU usage by mode	user / system / iowait etc. over time
Memory	total vs used over time
Network traffic	receive (rx) and transmit (tx) bytes/sec
Disk I/O	read and write bytes/sec

RADONAIX — API

Panel	Shows
Request rate	Requests per second across the API
Error rate (5xx)	Server-error requests per second (turns red past a threshold)
Latency p95 / p50	95th- and 50th-percentile response time
Request rate by path	Per-endpoint traffic (table legend)
Latency p50 / p95 / p99	Percentile latency trend
Responses by status code	2xx / 3xx / 4xx / 5xx breakdown over time

Both refresh every 30s and default to a 6-hour window (adjustable in the top-right of Grafana).

---

4. Run it on your laptop (quick check)

This is the fastest way to verify the dashboards. **Grafana is accessed directly on `:3000`** — you don't need nginx or the `/grafana/` sub-path locally.

Prerequisite — backend running (for the API dashboard)

```
cd /Users/manoj/Desktop/Revenue_assurance/backend
uvicorn app.main:app --host 127.0.0.1 --port 8000
# verify in another terminal:
curl -s localhost:8000/metrics | grep -c http_requests_total      # > 0
```

(The System dashboard works without this — it only needs node_exporter.)

Option A — Homebrew (native; real Mac metrics)

```
cd /Users/manoj/Desktop/Revenue_assurance/backend
brew install prometheus node_exporter grafana

# 1) system exporter on :9100
brew services start node_exporter

# 2) Prometheus with our committed config (its 127.0.0.1 targets are correct locally)
prometheus --config.file="$PWD/deploy/prometheus/prometheus.yml" \
  --storage.tsdb.path=/tmp/promdata --web.listen-address=127.0.0.1:9090 &

# 3) Grafana provisioning — datasource + dashboards (use ABSOLUTE paths)
mkdir -p /tmp/graf/provisioning/datasources /tmp/graf/provisioning/dashboards
cp deploy/grafana/provisioning/datasources/prometheus.yml /tmp/graf/provisioning/datasources/
cat > /tmp/graf/provisioning/dashboards/dashboards.yml <<'EOF'
apiVersion: 1
providers:
  - name: RADONAIX
    orgId: 1
    folder: RADONAIX
    type: file
    updateIntervalSeconds: 30
    allowUiUpdates: true
    options:
      path: /Users/manoj/Desktop/Revenue_assurance/backend/deploy/grafana/dashboards
      foldersFromFilesStructure: false
EOF

# 4) launch Grafana WITH the provisioning env vars (keep this terminal open)
GF_PATHS_PROVISIONING=/tmp/graf/provisioning \
GF_PATHS_DATA=/tmp/grafana-data \
GF_PATHS_LOGS=/tmp/grafana-logs \
GF_SECURITY_ADMIN_PASSWORD=admin \
grafana server --homepath "$(brew --prefix grafana)/share/grafana"
```

Open **http://localhost:3000** → log in `admin` / `admin` → **Dashboards** → **RADONAIX**. Check the targets are green at **http://localhost:9090/targets**.

Stop: `Ctrl-C` Grafana & Prometheus, then `brew services stop node_exporter`.

Option B — Docker (isolated; nothing installed on the Mac)

```
cd /Users/manoj/Desktop/Revenue_assurance/backend

cat > /tmp/prometheus.local.yml <<'EOF'
global: { scrape_interval: 15s }
scrape_configs:
  - job_name: radonaix_api
    metrics_path: /metrics
    static_configs: [ { targets: ["host.docker.internal:8000"] } ]
  - job_name: node
    static_configs: [ { targets: ["node-exporter:9100"] } ]
EOF

mkdir -p /tmp/graf/provisioning/datasources /tmp/graf/provisioning/dashboards /tmp/graf/dashb
cp deploy/grafana/provisioning/datasources/prometheus.yml  /tmp/graf/provisioning/datasources
cp deploy/grafana/provisioning/dashboards/dashboards.yml   /tmp/graf/provisioning/dashboards/
cp deploy/grafana/dashboards/*.json                        /tmp/graf/dashboards/
sed -i '' 's#http://127.0.0.1:9090#http://prometheus:9090#' /tmp/graf/provisioning/datasource

cat > /tmp/monitoring.compose.yml <<'EOF'
services:
  prometheus:
    image: prom/prometheus:latest
    volumes: ["/tmp/prometheus.local.yml:/etc/prometheus/prometheus.yml:ro"]
    ports: ["9090:9090"]
  node-exporter:
    image: prom/node-exporter:latest
    ports: ["9100:9100"]
  grafana:
    image: grafana/grafana:latest
    environment: { GF_SECURITY_ADMIN_PASSWORD: admin }
    volumes:
      - "/tmp/graf/provisioning:/etc/grafana/provisioning:ro"
      - "/tmp/graf/dashboards:/var/lib/grafana/dashboards:ro"
    ports: ["3000:3000"]
    depends_on: [prometheus]
EOF

docker compose -f /tmp/monitoring.compose.yml up
```

Open **http://localhost:3000** (`admin` / `admin`). **Stop:** `docker compose -f /tmp/monitoring.compose.yml down` .

> In Docker on a Mac, node-exporter reports the Docker VM (not the Mac). Fine for verifying the dashboards render; the API dashboard still shows your real backend.

See the UI screen too

```
cd /Users/manoj/Desktop/Revenue_assurance/ui/radon-ai-vision
echo "VITE_GRAFANA_URL=http://localhost:3000" >> .env.local
npm run dev
```

Log in as **admin** → **System Monitoring** appears in the sidebar; the button and cards open the local Grafana dashboards in a new tab.

---

5. Run it in production (server)

On the RHEL/Rocky VM the stack runs as **systemd services** behind the existing nginx, served at `https://<host>/grafana/`. Full step-by-step is in `backend/docs/MONITORING.md`; the shape:

1. **node_exporter** → binary into `/opt/monitoring/node_exporter/`, install `deploy/systemd/node_exporter.service`, `systemctl enable --now node_exporter` (binds `127.0.0.1:9100`).
2. **Prometheus** → binary into `/opt/monitoring/prometheus/`, copy `deploy/prometheus/prometheus.yml` to `/etc/prometheus/`, install `deploy/systemd/prometheus.service`, enable it (binds `127.0.0.1:9090`).
3. **Grafana** → install from the Grafana RPM repo; set `root_url=https://<host>/grafana/` and `serve_from_sub_path=true` (see `deploy/grafana/grafana.ini`); set the admin password via the `GF_SECURITY_ADMIN_PASSWORD` systemd override; copy the provisioning files + dashboards; `systemctl enable --now grafana-server` (binds `127.0.0.1:3000`).
4. **nginx** → the `/grafana/` proxy block is already in `deploy/nginx/radonaix.conf`; `nginx -t && systemctl reload nginx`.

Verify: `systemctl is-active node_exporter prometheus grafana-server` → all `active`; Prometheus *Targets* shows `radonaix_api` + `node` **UP**; open `https://<host>/grafana/`.

---

6. Access from the UI

A **System Monitoring** screen was added to the application (sidebar → *System Monitoring*). It is gated by the existing **Settings** permission (admins; RA Manager read-only), and it **links out** to Grafana in a new tab — it does not embed it. The button defaults to the relative `/grafana` path (same origin in production); set `VITE_GRAFANA_URL` at build time to point elsewhere.

---

7. Security

- Prometheus (9090), node_exporter (9100) and Grafana (3000) all bind to **127.0.0.1** and are never opened on the firewall — Grafana is reached only through nginx TLS at `/grafana/`.

- Grafana has its own login; anonymous access and iframe embedding are **off** by default.
- Change the Grafana admin password from `admin`; rotate via the systemd environment override.
- The backend `/metrics` endpoint is internal — nginx never proxies it (it scrapes the app directly).

---

8. Troubleshooting

Symptom	Cause	Fix
Logged into Grafana but **no dashboards**	Grafana running without our provisioning env vars, or an older instance is holding `:3000`	`pkill -f grafana`, then relaunch with the `GF_PATHS_PROVISIONING=…` prefix on the same command
RADONAIX folder empty	The dashboards provider `path` points at a non-existent folder (e.g. a wrong `$PWD`)	Use an **absolute** path in `dashboards.yml` → `…/backend/deploy/grafana/dashboards`
Panels show **"No data"**	The matching target is down	Open `:9090/targets` — `node` needs node_exporter on :9100; `radonaix_api` needs the backend on :8000
API dashboard empty but System works	Backend not running / `/metrics` unreachable	Start the backend; `curl localhost:8000/metrics` should return text
Datasource error in Grafana	Prometheus URL unreachable from Grafana	Local Homebrew: `http://127.0.0.1:9090`; Docker: `http://prometheus:9090`

---

9. Quick reference

Thing	Value
Grafana (local)	http://localhost:3000 · `admin` / `admin`
Grafana (prod)	https://<host>/grafana/
Prometheus UI	http://localhost:9090 (`/targets` for scrape health)
App metrics	http://localhost:8000/metrics
Scrape interval	15s · retention 30 days
Config files	`backend/deploy/prometheus/`, `backend/deploy/grafana/`, `backend/deploy/systemd/`
Server runbook	`backend/docs/MONITORING.md`